Project Report

GooglePay Inspired Expense Sharing System

1. Introduction

The Expense Sharing System is a Python-based backend application designed to streamline the management of shared expenses among a group of individuals. Whether for roommates, travel groups, or event organizers, tracking who paid for what and who owes whom can be computationally complex. This project solves that problem by meticulously tracking individual contributions, calculating equal splits, and providing an optimized algorithm for debt settlement.

2. Core Features

- **Participant Management:** The system easily initializes a ledger for an arbitrary number of friends, starting everyone at a zero balance.
- **Expense Logging & Splitting:** When an expense is recorded, the system automatically divides the total cost equally among the designated participants. It appropriately debits the participants and credits the payer.
- **Transaction Auditing:** Using the `pandas` library, every expense is saved as a structured record (including Payer, Amount, and Participants involved) to guarantee a transparent audit trail.
- **Debt Settlement (Simplification):** The application calculates the most efficient way to clear out all balances by minimizing the number of transactions required between debtors and creditors.

3. Technical Architecture

The codebase is encapsulated in a highly modular object-oriented structure using the `ExpenseSharing` class.

Libraries Utilized:

- **Pandas:** Used for creating and maintaining a robust `DataFrame` to track the transaction history.
- **NumPy & Matplotlib:** Included in the environment for potential future integration (e.g., numerical aggregations and visual charts of spending habits).

3.1. Method Breakdown

- **`__init__(self, friends):`** Initializes the core data structures, including a balance dictionary for quick O(1) lookups and an empty Pandas DataFrame for transaction history.
- **`add_expense(self, payer, amount, participants):`** Calculates the `split_amount` and updates the ledger. It subsequently appends a new row to the `transactions` DataFrame, preserving the historical integrity of group spending.
- **`calculate_settlement(self):`** The algorithmic core of the program. It partitions the users into a *creditors* list (those with positive balances) and a *debtors* list (those with negative balances).

4. Settlement Algorithm Analysis

The `calculate_settlement` method implements a **Greedy Algorithm** to resolve debts. By popping a debtor and a creditor from their respective stacks, it processes a transaction equal to the minimum of what the debtor owes and what the creditor is owed.

```
payment = min(debt_amount, credit_amount)
print(f"{debtor} owes {creditor}: rs. {payment:.2f}")
```

If the settled payment does not fully cover either the total debt or the total credit, the remainder is calculated and the participant is pushed back onto the stack. This iterative approach ensures that the total number of physical money transfers is minimized, effectively "simplifying" the group's debts.

5. Future Enhancements

While the current system provides a robust mathematical backend, it can be extended in several ways:

- **Unequal Splits:** Modifying the `add_expense` function to handle percentage-based or exact-amount splits.
- **Data Visualization:** Utilizing the imported `matplotlib.pyplot` library to render pie charts of individual spending and bar charts of total group expenditures.
- **Persistent Storage:** Exporting the `pandas` DataFrame to a CSV or Excel file so that transaction history can persist between sessions.
